

# Artificial Intelligence

## Lesson 5: Uninformed Search

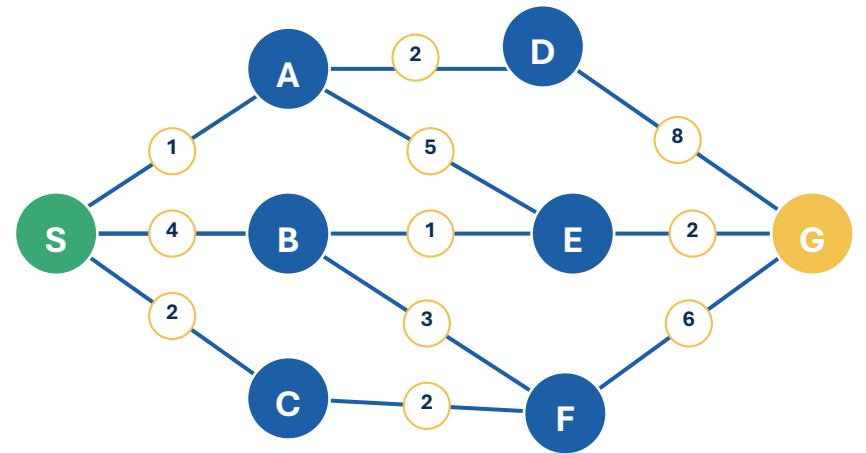
Same map, different frontier, different behavior

Start

Frontier

Expand

Goal



Search is systematic exploration controlled by a selection rule.

# Retrieval warm-up

Lesson 4 gave the search algorithm a problem representation.

**We have:**

Start state

Actions

Goal test

Path costs

**What information is still missing before the system can find a solution?**

## Question 1

Which state is examined next?

## Question 2

What gets remembered?

## Question 3

When do we stop?

## Question 4

How do we reconstruct the path?

# Today's mission

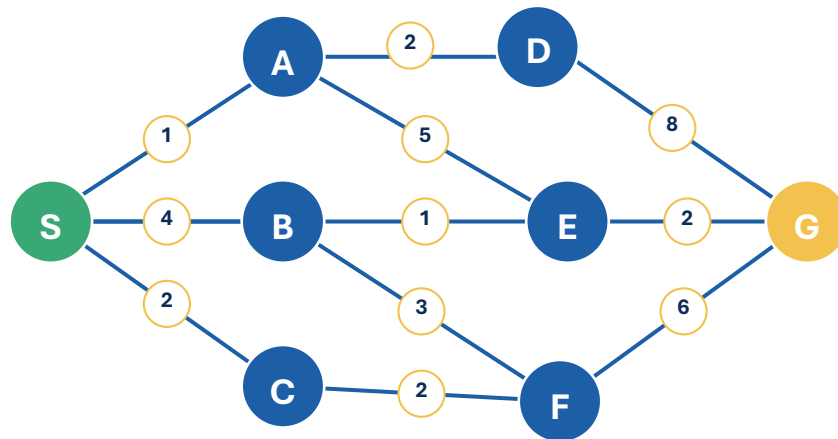
By the end of Lesson 5, students should be able to...

- 1 **Describe how DFS, BFS, and UCS explore state spaces**
- 2 **Compare search strategies based on completeness and optimality**
- 3 **Explain how frontier management affects algorithm behavior**

**Performance target: trace a small search, identify the next node, and justify the algorithm choice.**

# Running scenario: relay recovery

The rover must reach a failed communications relay through waypoint routes.



## Mission setup

- Start: S
- Goal relay: G
- Edge labels: energy cost
- Ties: break alphabetically
- Same graph, three search behaviors

**Should the rover prefer fewer moves or lower energy?**

# What is a search node?

A node stores enough information to recover how a state was reached.

| Part   | Purpose           |
|--------|-------------------|
| State  | current situation |
| Parent | previous node     |
| Action | move from parent  |
| Depth  | number of actions |
| $g(n)$ | cost from start   |

## State vs. node

State: one situation in the problem.

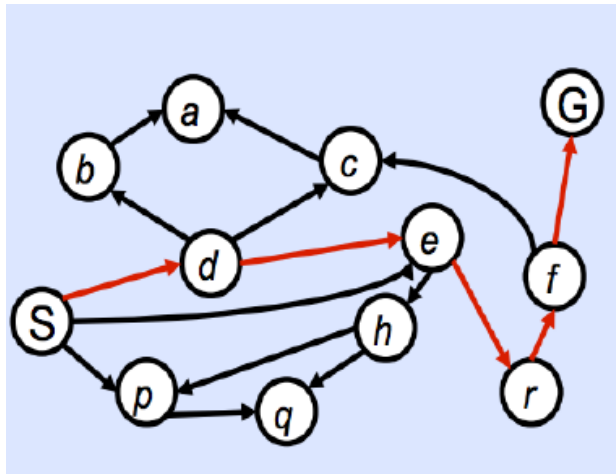
Search node: a record of one path that reaches that state.

The same state can appear through different paths with different costs.

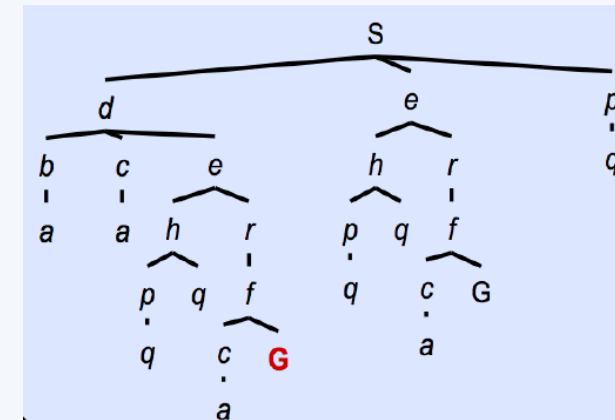
# State space, search tree, repeated states

One situation may be reached through several paths.

## State space



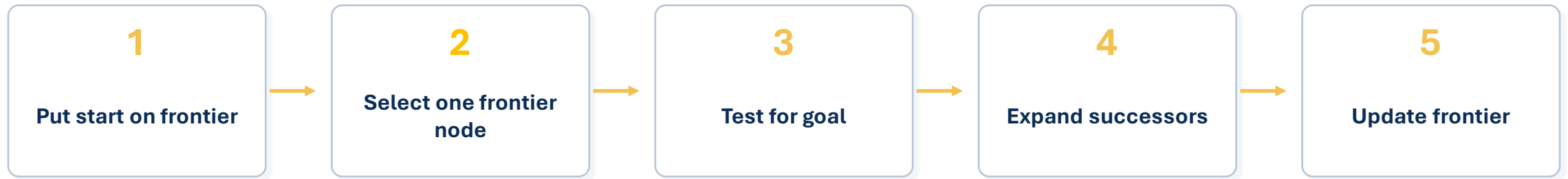
## Search tree



An explored set helps prevent cycles and repeated expansion.  
For UCS, a cheaper path may need to replace a more expensive one.

# Search is repeated selection

Most search algorithms share the same basic loop.



**The algorithm changes when the selection rule changes.**



# Search pseudo-algorithms

Maintaining a fringe/frontier is the key

closed aka expanded

```
function TREE-SEARCH(problem, fringe) return a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    for child-node in EXPAND(STATE[node], problem) do
      fringe ← INSERT(child-node, fringe)
    end
  end
```

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      for child-node in EXPAND(STATE[node], problem) do
        fringe ← INSERT(child-node, fringe)
      end
    end
  end
```





(aka fringe)

# The frontier controls the algorithm

The search loop stays similar; the frontier rule changes behavior.

| Algorithm | Frontier structure | Select next         |
|-----------|--------------------|---------------------|
| BFS       | Queue              | oldest / shallowest |
| DFS       | Stack              | newest / deepest    |
| UCS       | Priority queue     | lowest $g(n)$       |

**Same graph. Same goal. Different frontier. Different expansion order.**

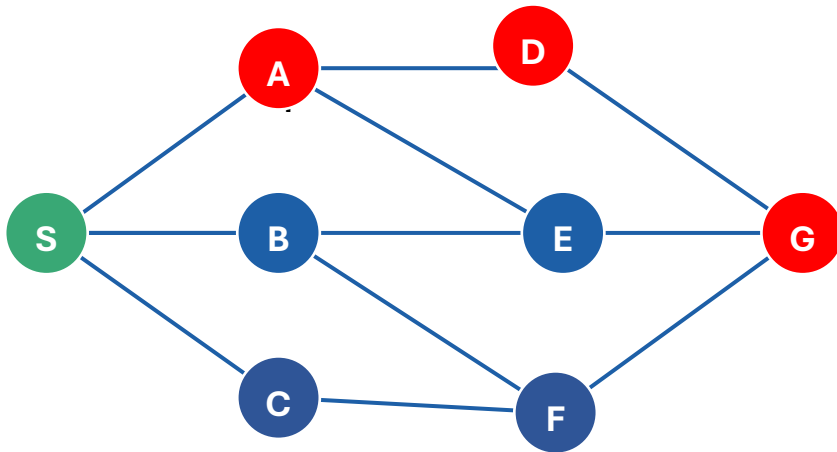
# Algorithms can be compared

How to determine which algorithm is best for a problem

| Property        | Definition                                                                                                                      |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------|
| Completeness    | If there exists a solution to the search problem, is the strategy guaranteed to find it given infinite computational resources? |
| Optimality      | Is the strategy guaranteed to find the lowest cost path to a goal state?                                                        |
| Memory Use      | How much memory is needed to store the searched nodes?                                                                          |
| Appropriate Use | In what case should the algorithm be used?                                                                                      |

# Breadth-First Search: explore by layers

BFS expands the shallowest available node first.



**Frontier behavior**

**Queue  
first in, first out**

**BFS does not know edge costs unless every step cost is effectively equal.**

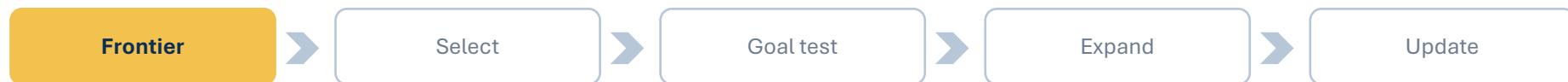
# BFS queue mechanics

The oldest frontier node is expanded next.



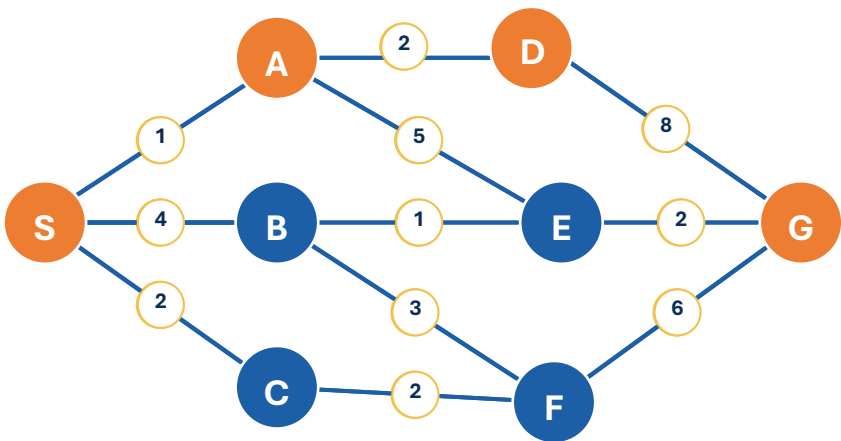
**Example update:**

**Pop A → add D, E → frontier becomes [B, C, D, E]**



# Guided BFS trace

Track expansion order, frontier, and returned path separately.



| Step | Expanded | Frontier                                    |
|------|----------|---------------------------------------------|
| 0    | —        | [(S)]                                       |
| 1    | S        | [(S,A), (S,B), (S,C)]                       |
| 2    | A        | [(S,B), (S,C), (S,A,D), (S,A,E)]            |
| 3    | B        | [(S,C), (S,A,D), (S,A,E), (S,B,E), (S,B,F)] |
| ...  | partner  | complete ...                                |

Do not confuse expanded order, frontier contents, and solution path.

# What BFS guarantees

BFS is strongest when every action has the same cost.

## Complete?

Yes, with finite branching (i.e., a solution exists)

## Optimal?

No, unless all edge costs are equal

## Memory

Often high: stores a full frontier layer ( $O(b^s)$ )

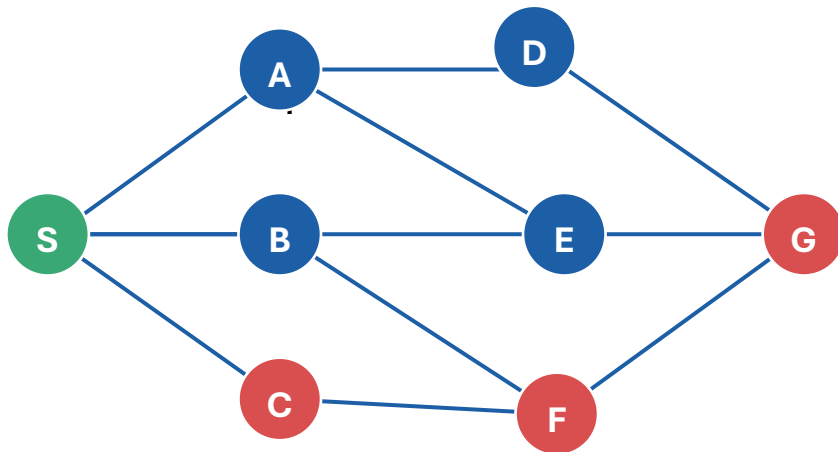
## Best use

fewest actions, shallow goals

**Key misconception: shallowest does not necessarily mean cheapest.**

# Depth-First Search: follow one branch

DFS expands the most recently added frontier node.



## Frontier behavior

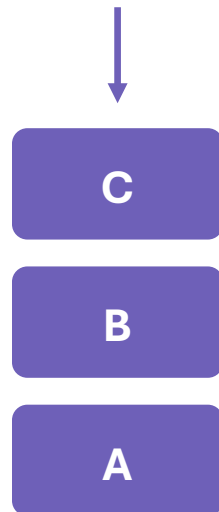
**Stack**  
**last in, first out**

DFS can reach a solution quickly, but it may commit to a poor branch first.

# DFS stack mechanics

The newest frontier node is expanded next.

Top of stack



## Successor-order convention

If successors are placed on the frontier alphabetically, DFS removes the most recently added one.

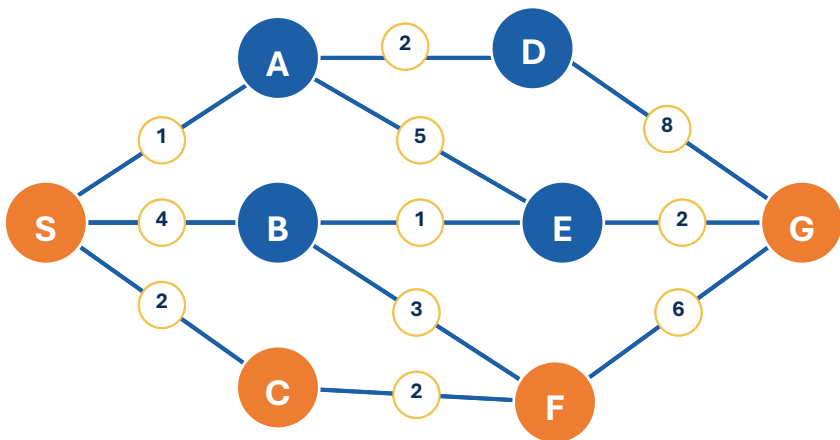
That can make the last alphabetical successor expand first.

**LIFO is simple; tracing it consistently is the hard part.**



# Guided DFS trace

Use the same graph; only the frontier rule changes.



| Step | Expanded | Frontier                |
|------|----------|-------------------------|
| 0    | —        | [(S)]                   |
| 1    | S        | [(S,A), (S,B), (S,C)]   |
| 2    | C        | [(S,A), (S,B), (S,C,F)] |
| ...  | partner  | complete ...            |

With this ordering, DFS returns the first deep solution it finds.

# What DFS guarantees

DFS trades weaker guarantees for lower memory.

## Complete?

Not guaranteed with cycles or infinite depth

## Optimal?

No (it will find the leftmost solution without regard for path costs)

## Memory

Often low: stores a branch and alternatives, so  $O(bm)$

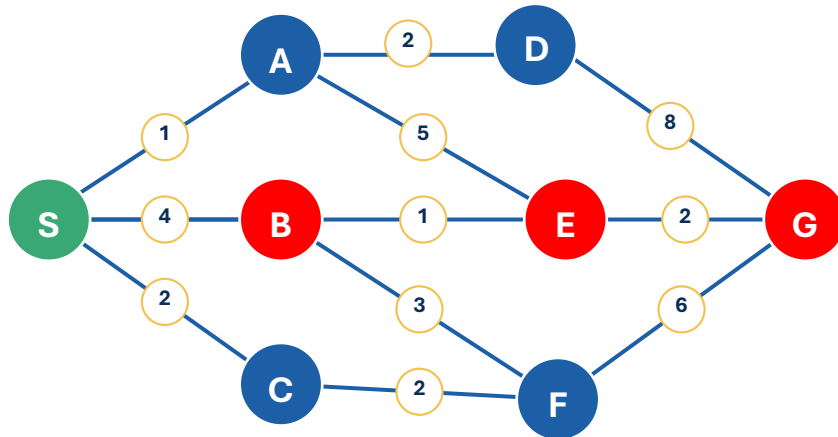
## Best use

deep search when any solution is acceptable

**DFS is not “bad BFS.” It is a different tradeoff.**

# Uniform-Cost Search: follow accumulated cost

UCS expands the frontier node with the smallest path cost  $g(n)$ .



## What UCS knows

$g(n)$  = cost already accumulated  
from the start to node  $n$

UCS is still uninformed: it does not  
estimate the remaining cost to the goal.

# UCS priority-queue mechanics

The frontier is ordered by lowest accumulated path cost.

| Node | $g(n)$ |
|------|--------|
| A    | 4      |
| B    | 2      |
| C    | 7      |



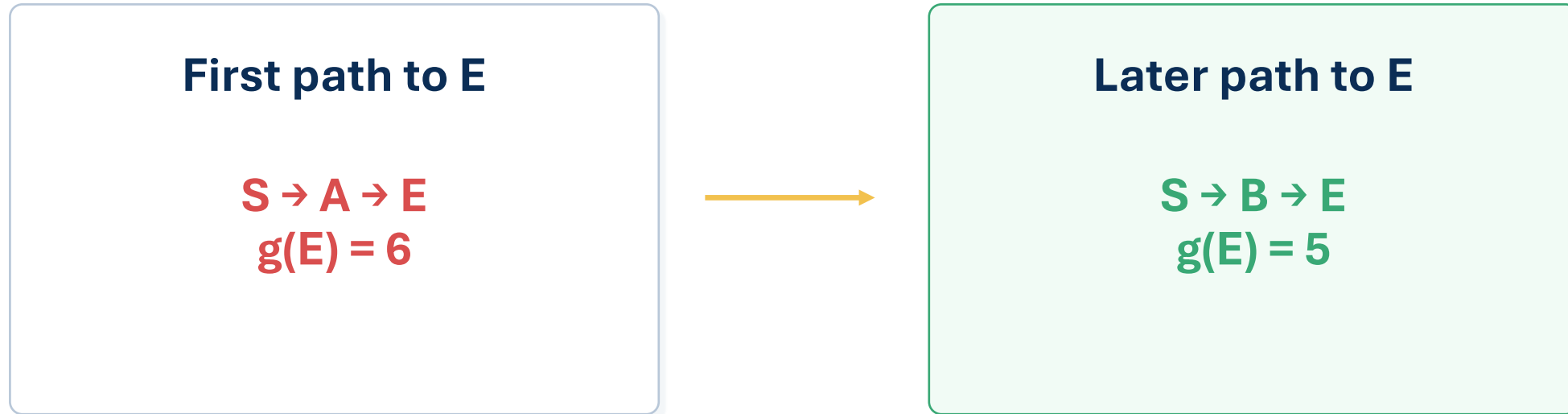
**UCS selects B**

because  $g(B)$  is smallest

**Not oldest. Not shallowest. Not closest-looking. Lowest accumulated cost.**

# A cheaper path can replace an expensive one

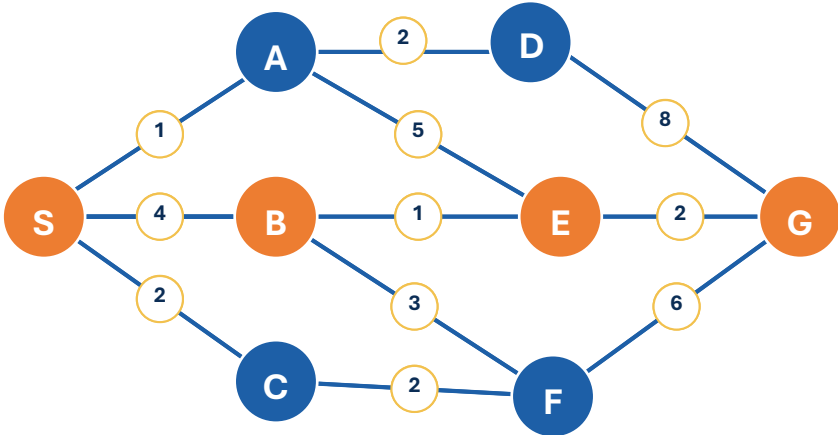
UCS must keep the best known path to a frontier state.



**Update E: lower cost changes its priority and parent path.**

# Guided UCS trace

List expansion order, frontier updates, final path, and total cost.



| Step | Expanded | Frontier after update                                   |
|------|----------|---------------------------------------------------------|
| 0    | —        | $[(S), 0]$                                              |
| 1    | S        | $[(S,A), 1), ((S,C), 2), ((S,B), 4)]$                   |
| 2    | A        | $[(S,C), 2), ((S,A,D), 3), ((S,B), 4), ((S,A,E), 6)]$   |
| 3    | C        | $[(S,A,D), 3), ((S,B), 4), ((S,C,F), 4), ((S,A,E), 6)]$ |
| ...  | partner  | finish trace                                            |

Goal rule: return G when it is removed as the lowest-cost frontier node.

# What UCS guarantees

UCS is designed for least-cost solutions.

## Complete?

Yes, with positive edge costs and finite branching

## Optimal?

Yes, least total path cost (assume all nonnegative edge costs)

## Memory

Potentially high, dependent on number of nodes at level of cheapest solution

## Best use

varying route costs; cost matters

**UCS may take more steps if doing so reduces total cost.**

# Same graph, three behaviors

The frontier rule changes the expansion order and returned route.

| Algorithm | Explores         | Likely return       | Tradeoff                  |
|-----------|------------------|---------------------|---------------------------|
| BFS       | layer by layer   | fewest actions      | may ignore cost           |
| DFS       | branch by branch | first deep solution | low memory, no optimality |
| UCS       | cost by cost     | least path cost     | uses $g(n)$               |

**The problem did not change. The frontier did.**

That is why frontier management is algorithm behavior.



# Completeness, optimality, and memory

Guarantees depend on assumptions.

| Algorithm | Selection rule   | Complete?             | Optimal?                | Memory           |
|-----------|------------------|-----------------------|-------------------------|------------------|
| BFS       | shallowest       | yes, finite branching | only w/equal step costs | high             |
| DFS       | newest / deepest | not guaranteed        | no                      | low              |
| UCS       | lowest $g(n)$    | yes, positive costs   | yes, nonnegative costs  | potentially high |

**Do not memorize guarantees without their conditions.**

# Choose the algorithm

Match the problem objective to the frontier rule.

## Fewest actions

Every repair action takes one hour; minimize number of repairs.

**BFS**

## Memory limited

A deep configuration may work; any solution is acceptable.

**DFS**

## Least cost

Routes have different fuel costs; minimize total fuel.

**UCS**

**Justification matters: name the property that makes the algorithm fit.**

# Diagnose the frontier

Data structure reveals search behavior.

**Queue**

**[A, B, C]  
select A**

**BFS**

**Stack**

**top: C, B, A  
select C**

**DFS**

**Priority queue**

**A:5, B:2, C:4  
select B**

**UCS**

**Code interpretation: same loop, different frontier type.**

# Exit Ticket

Rebuild the search comparison from memory.

## Complete

**BFS** → \_\_\_\_\_ → **shallowest**

**DFS** → \_\_\_\_\_ → **newest/deepest**

**UCS** → \_\_\_\_\_ → **lowest  $g(n)$**

The frontier determines which node is expanded next.

## Apply

Frontier snapshot:

A: depth 1,  $g=5$ , added first

B: depth 2,  $g=2$ , added second

C: depth 1,  $g=4$ , added third

Which node does BFS select? DFS? UCS?

**Next class: Informed Search — what changes when the system has an estimate of what remains?**